Problem Set 3

Hoa T. Vu

Algorithms

Problem 1: Counting Pairs of 1s and 0s. Given an array $a[0..n-1]$ of 0s and 1s, count the number of pairs (i, j) with $i < j$ such that $a[i] = 1$ and $a[j] = 0$.

(a) What is the running time of the naive solution using two nested loops?

(b) Design a divide-and-conquer algorithm that runs in $O(n \log n)$.

(c) Can you design an algorithm that runs in $O(n)$ time?

(d) Implement your algorithm and verify its correctness using the test files.

Hint: keep a running count of how many 1s you've seen so far as you scan the array once from left to right.

Problem 2: Coin Change. *(LeetCode 322 – Coin Change)* Given coin denominations, find the **minimum number of coins** to make a target amount. Each coin may be used any number of times.

Example: coins $= [1, 5, 6, 9]$, amount $= 11$. Greedy (largest first) gives $9 + 1 + 1 = 3$ coins. Can you do better?

Hint: think about subproblems. If you know the minimum coins for every amount $< a$, how can you compute the minimum coins for amount a?

(a) Define the subproblem. Write the recurrence.

(b) Implement `coin_change(coins, amount)`.

(c) What is the running time?

Problem 3: 3-Sum. *(LeetCode 15 – 3Sum)* Given an array of integers and a target T, find all unique triplets (a, b, c) such that $a + b + c = T$.

Example: `[-1, 0, 1, 2, -1, -4]`, $T = 0 \Rightarrow$ triplets: $(-1, -1, 2)$ and $(-1, 0, 1)$.

(a) Write a brute-force $O(n^3)$ solution using three nested loops.

(b) Write an $O(n^2 \log n)$ time solution. Hint sort the array. Then for every pair $a[i]$ and $a[j]$ where $i \neq j$, how do you quickly find if there is $k \neq i, j$ such that $a[i] + a[j] + a[k] = T$?

Problem 4: Counting Inversions.

An **inversion** in a list is a pair (i, j) with $i < j$ but `arr[i] > arr[j]`.

```
arr = [3, 1, 2, 5, 4]
# inversions: (3,1), (3,2), (5,4)  ->  count = 3
```

(a) Write a brute-force solution using two nested loops. What is its time complexity?

(b) Write and implement an $O(n \log n)$ solution.

(c) Implement your algorithm and verify its correctness using the test files.

Hint: think about merge sort. Try counting the number of inversions in the left and right halves recursively along with sorting them.